

Итоговый проект

1) Данные

- Выбрать набор данных: **CSV / JSON / текстовые логи** (объёмные, а также требующие очистки и нормализации).
- В README указать **источник** и кратко описать **структуру** (поля и их смысл).

2) Аналитика в pandas

- Выполнить **минимум 5** осмысленных шагов обработки в pandas, например:
 - ❖ очистка/нормализация (типы, даты, пропуски),
 - ❖ фильтрация,
 - ❖ создание новых колонок
 - ❖ и т.д.
- Сохранить результат: **очищенный датасет и/или отчёт** в CSV/JSON.

3) Поточковая обработка без pandas

Реализовать ленивый пайплайн чтения/предобработки данных, без загрузки всего в память.

В составе обязательно:

- **итерируемый объект** (например DataSource/LogReader);
- **кастомный итератор или генератор** с yield;
- **итератор-фильтр/преобразователь**, который оборачивает входной итератор и работает лениво.

4) ООП-структура

- Проект оформлен как пакет/модули.
- Минимум **3 содержательных класса**, например:
 - ❖ источник/загрузчик данных,
 - ❖ обработчик/пайплайн,
 - ❖ компонент отчёта/экспорта **или** набор правил/стратегий.

5) Исключения

- Определить минимум **2 пользовательских исключения**.
- В коде должны быть места, где эти исключения **выбрасываются**, и где они **корректно обрабатываются**.

6) Декоратор

- Использовать **собственный декоратор** в проекте по назначению, например:
 - повтор операции при ошибках,
 - логирование/тайминг,
 - валидация входных параметров.

7) Паттерн проектирования

- Применить минимум **1 паттерн** в рабочем коде.
- В README кратко указать: **какой паттерн** и **где используется**.

8) Тестирование (pytest)

- Минимум **8 тестов**.
- Обязательно использовать:
 - `fixture`,
 - `parametrize`,
 - `pytest.raises`,
 - пользовательскую `mark`.

9) Сдача проекта

- Репозиторий с кодом, тестами и README.
- В README: как запустить обработку, как запустить тесты, какие файлы создаются на выходе.
- Информативная, понятная презентация с объяснением работы проекта и демонстрацией результатов.